⚡ Wind Power Forecasting Challenge

> *"Two models. One grid. Real trading decisions."*

🌪️ The Situation

You're joining the forecasting team at a power trading company. We trade wind energy in the day-ahead market, and accurate forecasts directly impact our bottom line. A 10 MW forecasting error across a portfolio? Multiply that by thousands of trading intervals and you're looking at serious imbalance costs that add up fast.

We've been running two production forecasting models:

Zephyr — Our workhorse. Consistently profitable, conservative, and smooth. Right most of the time when conditions are "normal." Fast, reliable, and loves predictable weather patterns. Struggles when weather patterns get... interesting.

Boreas — Our maverick. More aggressive, higher variance, but catches the disruptions Zephyr misses. Particularly good at predicting when shit's about to get weird (which, in wind forecasting, is often). The catch? Boreas can also be spectacularly wrong. When it's wrong, it's *confidently* wrong with large errors.

Here's the problem: **Zephyr is profitable except on the days when Boreas predicts better.** And we've been manually switching between them like cavemen, leaving money on the table.

Your mission: Build a meta-forecaster that **automatically picks the right model at the right time** — or combines them intelligently — to minimize our forecast error and maximize our trading performance.

📦 What You're Getting

One year of hourly wind power data from our zone:

```
 forecast_train.csv    # 80% historical — features, both forecasts, actuals
forecast_test.csv     # 20% holdout — features and forecasts only
```

Data Schema

Column	Description
`timestamp`	Hourly UTC timestamps
`feature_0` ... `feature_19`	20 weather-related features from our NWP models

Column	Description
`forecast_zephyr`	Zephyr's wind power prediction (MW)
`forecast_boreas`	Boreas's wind power prediction (MW)
`actual_mw`	Actual metered generation *(train only)*

Your deliverable: Predictions for the test set in CSV format with `timestamp` and `predicted_mw` columns.

🎯 Success Criteria

Mean Absolute Error (MAE) in MW — because every megawatt of error costs us real money.

The Reality Check

- **Current naive approach**: Simple average of both models → baseline MAE
- **Theoretical best (Oracle)**: Always pick the better forecast → theoretical minimum MAE
- **Your target**: Build something deployable that gets us closer to actual_mw

We're not expecting miracles. A 10-15% improvement over naive averaging would be genuinely valuable. 20%+ would be promotion-worthy.

🧠 What We're Evaluating

This isn't a Kaggle competition. We're hiring for a real trading desk. That means we care about:

1. Understanding & Reasoning (40%)

- *Why* does your approach work?
- What patterns did you identify in model failures?
- Can you explain your logic to a non-technical trader at 6am when the model is doing something weird?
- Do you understand the problem, or did you just throw XGBoost at it?

2. Code Quality & Engineering (30%)

- Is your code production-ready or "works on my machine"?
- Can it run at 4am when you're asleep and the on-call engineer is panicking?
- Are there tests? Logging? Error handling?
- Would we trust this to make million-euro trading decisions?

3. Deployment & Operations (20%)

- How would this run in production?
- Is your work versioned? Reproducible? Rollback-able?
- What happens when we need to retrain? When a model drifts?

- Can you show us Docker, config management, model versioning, CI/CD?

4. ML/Feature Engineering (10%)

- Yes, it matters. But a mediocre model with great engineering beats a great model with shit engineering.
- Clever insights > 47 hyperparameter combinations.

🏗️ Approach Archetypes

The Data Scientist 📊

"Let me explore this in Jupyter first..."

- Loads data, plots everything, builds intuition
- Engineers features, trains models, iterates in notebooks
- Exports predictions to CSV
- Documents findings in markdown

Good for: Quick insights, experimentation, proving concepts
Risk: Hard to reproduce, version, or deploy

The ML Engineer ⚙️

"If it doesn't have a Dockerfile, it doesn't exist."

- Structured project (`src/` , `models/` , `data/` , `configs/`)
- Versioned datasets (DVC) and models (MLflow)
- Config-driven pipelines (no hardcoded paths or magic numbers)
- Docker + requirements.txt + reproducible environment
- CLI for train/predict/evaluate
- Unit tests for critical logic

Good for: Production readiness, team collaboration, long-term maintenance
Risk: Takes longer to set up (but worth it)

The Trading Technologist 🚀

"Production isn't a destination, it's a lifestyle."

All of the above, plus:

- Model serving API (FastAPI/Flask)
- Monitoring & alerting (when model degrades)
- Automated retraining pipeline
- Cost analysis (compute vs. forecast improvement)
- Documentation that doesn't make people cry
- Makes us want to put this in production Monday morning

Good for: Impressing us, getting hired, not getting paged at 3am
Risk: Might be overkill for a take-home (but we'd be impressed)

📤 Submission Requirements

Send us a ZIP or GitHub repo containing:

Required:

1. **predictions.csv** — Your test set predictions (`timestamp` , `predicted_mw`)
2. **README.md** — Your approach, reasoning, and architecture decisions
3. **Code** — Notebooks, scripts, modules — whatever you built
4. **Executive Brief** (single-page PDF)
    - Visual report for the trading desk manager
    - Key findings from your analysis
    - Your solution approach in plain language
    - Quantified improvement vs baselines
    - Operational recommendations

Bonus Points:

- Deployment artifacts (Docker, API, model serving)
- Model versioning & experiment tracking
- CI/CD configuration
- Monitoring/alerting strategy
- Anything (be creative)

⚠️ Ground Rules

1. **No lookahead bias** — don't use future information to predict the past;
2. **External data allowed** — but unnecessary; everything you need is here; our data is synthetic;
3. **Be creative** — ensemble? Meta-learning? Time-series analysis? Rules-based logic? Do what works;
4. **LLMs are fine** - just understand/validate/test inputs and outputs.

💭 Our Philosophy

We don't care if you have a PhD in Machine Learning or dropped out of high school to trade crypto. We care if you can:

1. **Think critically** about complex problems
2. **Write code** that doesn't make people sad
3. **Ship systems** that work at 4am on a Sunday

A model with 1% better accuracy that crashes every deployment is worthless. A beautifully architected system with 50% worse performance is also worthless.

We're looking for the balance: **good enough accuracy** + **production-ready engineering** + **clear thinking**.

🎪 Final Words

Remember: Both Zephyr and Boreas were built by smart people. They're not bad models. They're just... *situational*. Your advantage is hindsight, creativity, and the ability to combine their strengths while avoiding their weaknesses.

Or you could just flip a coin. That's also a valid baseline.

Good luck, and may your gradients be ever in your favor! ⚡

P.S. — We estimate this should take 4-8 hours for an experienced candidate. If you're spending significantly more, you're probably over-engineering — just ship what you have. Done is better than perfect.

P.P.S. — If you're reading this line, you're the kind of attentive person who might also notice patterns in model performance over time. Just saying. 👀